



Intelligent Heuristics in Games

Introduction to Artificial Intelligence

Yangguang Shi

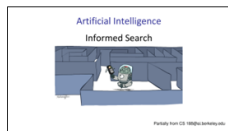
School of Computing Science and Technology
Shandong University

May 2025



山东大学

Week 4: Informed Search



Search Heuristics

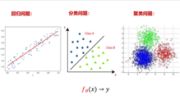


Heuristic Search

Week 5: Machine Learning



机器学习-解决什么问题?

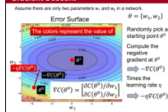


ML's Paradigm

Week 6: Deep Learning

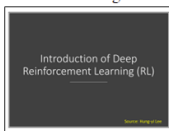


Gradient Descent



Gradient Descent

Week 7: Reinforcement Learning



Machine Learning = Looking for a Function



RL's Paradigm

Week 8: Generative AI



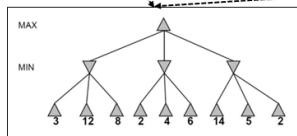
GAN GAN as a minimax game

$$V(\theta^{(D)}, \theta^{(G)}) = E_x[\log D(x)] + E_z[\log(1 - D(G(z)))]$$

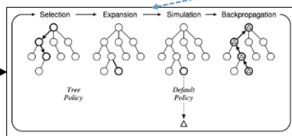
$$\min_{\theta^{(D)}} \max_{\theta^{(G)}} V(\theta^{(D)}, \theta^{(G)})$$

- V depends on the parameters of both players, but each player can only change its own parameters: a game
- D wishes to maximize V using only parameters $\theta^{(D)}$
- G wishes to minimize V using only parameters $\theta^{(G)}$
- The solution to a game is Nash equilibrium: $D(x) = 1/2$

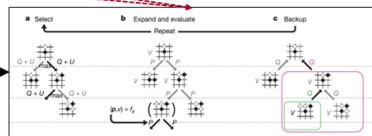
Minimax Game



Alpha-Beta Search



MCTS+UCB



MCTS+UCB+DL



Table of Contents

- ▶ Chess: Deep Blue
- ▶ Go: AlphaGo Zero
- ▶ Poker: Deep CFR



Two-Player Zero-Sum Games

- An important application of AI search has been in the domain of **two-player games**

Two-Player Zero-Sum Games

- Players take turns
- Each game outcome or terminal state has a utility for each player (e.g., 1 for win, -1 for loss)
- The sum of both players' utilities is a constant





IBM Machine “Deep Blue”

- Until relatively recently (late 1990s), it was widely believed that hard problems of intelligence would never be solved by a computer
- Chess was often put forward as a good example
- In May 1997, an IBM machine “Deep Blue” defeated chess grandmaster Garry Kasparov





Deep Blue: Technical Overview

- Deep Blue relies on tried and trusted methods
 - Modeling chess as a two-player, turn-based **Perfect-Information** game
 - Applying the classic **Minimax** algorithm to search the **Game Tree**
 - Employing **Heuristic Evaluation** and **Alpha-Beta Pruning** to make the minimax search framework computationally feasible

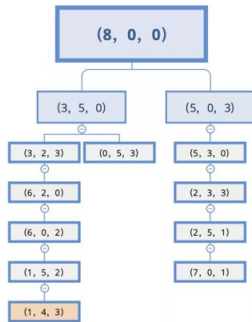




Adapting Search for Game Playing

- Games such as chess can be seen in terms of transitions between states
 - Transitions are moves; states are board configurations
- Single-Agent Search vs Game Playing:
 - Single-agent* search is to find a path of transitions from the start state to a goal state

Measure 4L Using 3 Buckets





Adapting Search for Game Playing

- Games such as chess can be seen in terms of transitions between states
 - Transitions are moves; states are board configurations
- Single-Agent Search vs Game Playing:
 - *Single-agent* search is to find a path of transitions from the start state to a goal state
 - Under the context of *game playing*, we lose control over the choice of move every other turn, and we do not know how the opponent will act





Minimax Search Framework

- Minimax strategy: Choose the move (transition) that gives the **best worst-case** payoff



Minimax Search Framework

- Minimax strategy: Choose the move (transition) that gives the **best worst-case** payoff
 - Every possible outcome has a value (utility)
 - (Worst-case) Opponent's rational action is to choose a move that minimizes the value of the outcome
 - (Best) My rational action is to choose a move that maximizes the value of the outcome



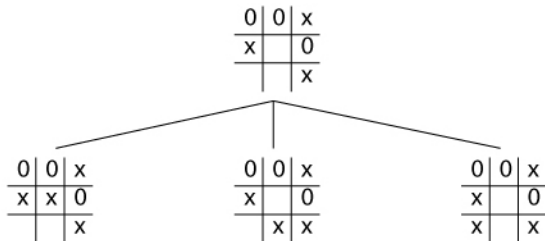
Minimax Search Framework

- Minimax strategy: Choose the move (transition) that gives the **best worst-case** payoff
 - Every possible outcome has a value (utility)
 - (Worst-case) Opponent's rational action is to choose a move that minimizes the value of the outcome
 - (Best) My rational action is to choose a move that maximizes the value of the outcome
- Three elements of the minimax method
 1. Expand the search tree all the way down to an outcome (win, lose, or draw)
 2. Obtain an evaluation of the relevant terminal state
 3. **Back-up** the evaluations level by level
 - When it is the opponent's turn, they will choose a move that achieves the worst outcome with the minimum value
 - Whenever it is our turn to move, we will choose the best



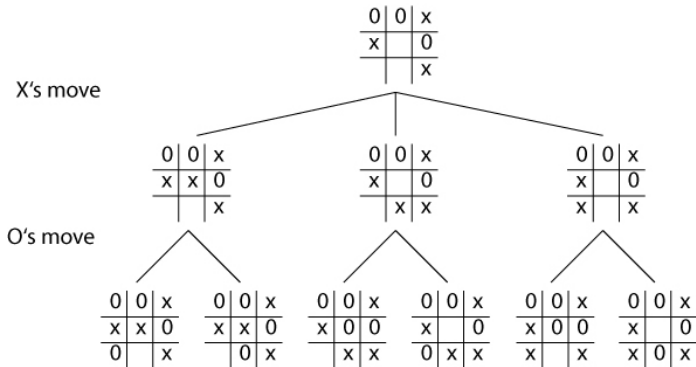
Example of Minimax Search

X's move



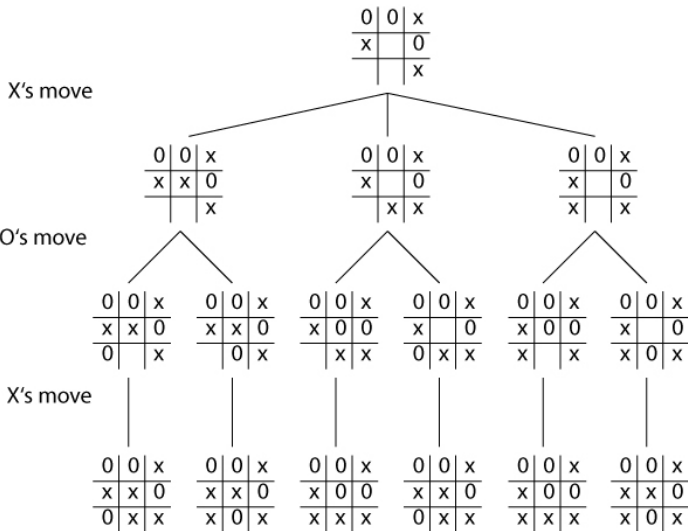


Example of Minimax Search



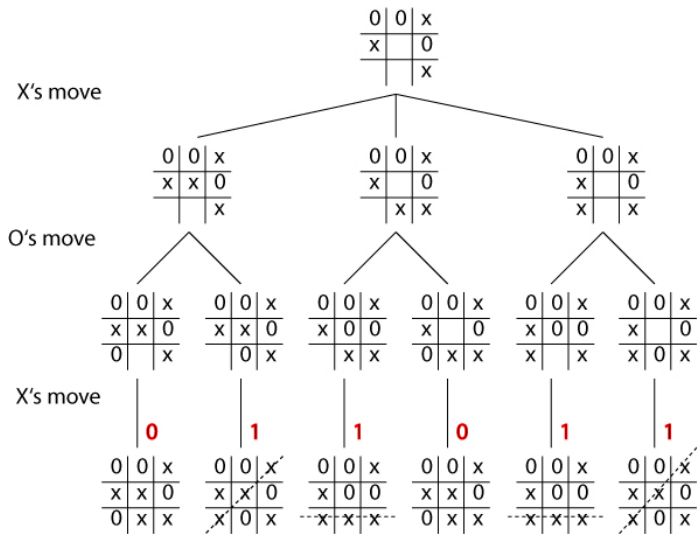


Example of Minimax Search



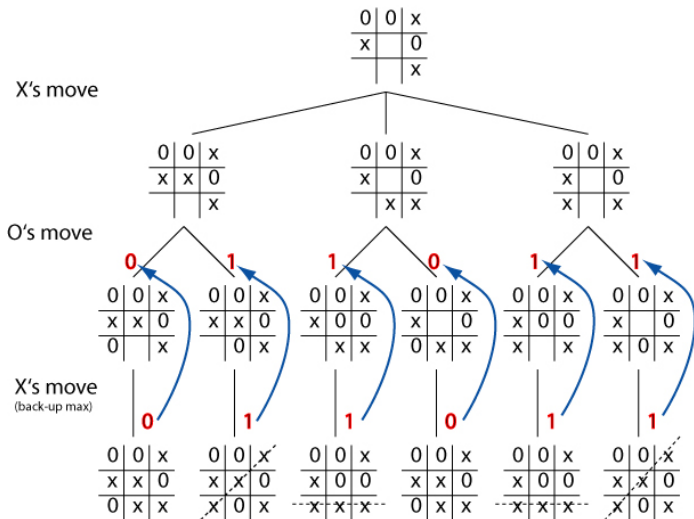


Example of Minimax Search



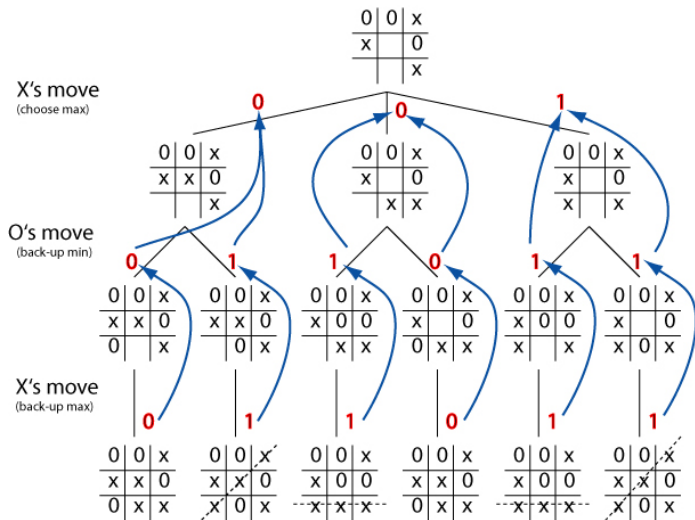


Example of Minimax Search





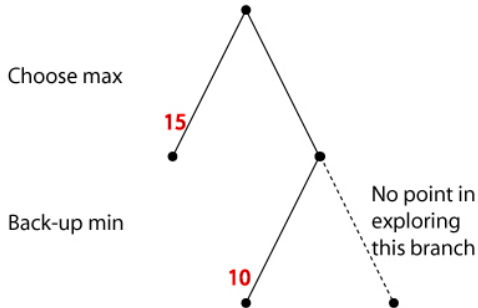
Example of Minimax Search





Alpha-Beta Pruning: Alpha-Cut

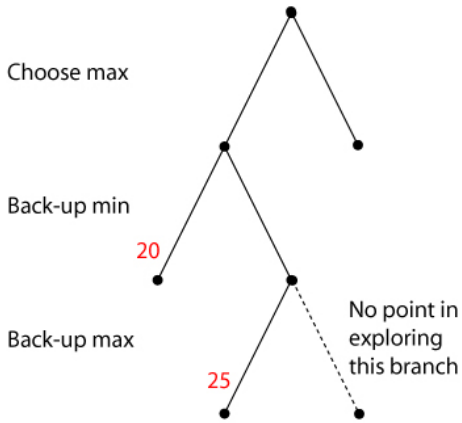
- Applying an **alpha-cutoff** means we stop searching a branch because we already have a better opportunity elsewhere





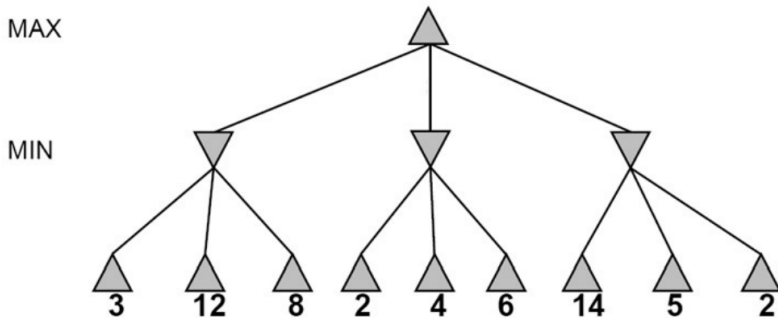
Alpha-Beta Pruning: Beta-Cut

- Applying a **beta-cutoff** means we stop searching a branch because the opponent has a better opportunity elsewhere





Alpha-Beta Pruning: Example





Alpha-Beta Pruning: Exercise

- POJ 1568: Find the Winning Move

Find the Winning Move

Language: Default

Time Limit: 3000MS **Memory Limit:** 32768K
Total Submissions: 2122 **Accepted:** 993

Description

4x4 tic-tac-toe is played on a board with four rows (numbered 0 to 3 from top to bottom) and four columns (numbered 0 to 3 from left to right). There are two players, x and o, who move alternately with x always going first. The game is won by the first player to get four of his or her pieces on the same row, column, or diagonal. If the board is full and neither player has won then the game is a draw.

Assuming that it is x's turn to move, x is said to have a forced win if x can make a move such that no matter what moves o makes for the rest of the game, x can win. This does not necessarily mean that x will win on the very next move, although that is a possibility. It means that x has a winning strategy that will guarantee an eventual victory regardless of what o does.

Your job is to write a program that, given a partially-completed game with x to move next, will determine whether x has a forced win. You can assume that each player has made at least two moves, that the game has not already been won by either player, and that the board is not full.

Input

The input contains one or more test cases, followed by a line beginning with a dollar sign that signals the end of the file. Each test case begins with a line containing a question mark and is followed by four lines representing the board; formatting is exactly as shown in the example. The characters used in a board description are the period (representing an empty space), lowercase x, and lowercase o. For each test case, output a line containing the (row, column) position of the first forced win for x, or '#####' if there is no forced win. Format the output exactly as shown in the example.

Output

For this problem, the first forced win is determined by board position, not the number of moves required for victory. Search for a forced win by examining positions (0, 0), (0, 1), (0, 2), (0, 3), (1, 0), (1, 1), ..., (3, 2), (3, 3), in that order, and output the first forced win you find. In the second test case below, note that x could win immediately by playing at (0, 3) or (2, 0), but playing at (0, 1) will still ensure victory (although it unnecessarily delays it), and position (0, 1) comes first.



State Space of Chess

- The size of the chess game tree grows exponentially as b^d
 - The branching factor b of a game tree is the average number of legal moves available from a typical position
 - Analysis and programming literature agree that $b \approx 35$
 - d is the average game length in half-moves
 - A half-move or “ply” represents a single player’s move
 - Statistical studies of master-level games show that the typical game lasts about $d \approx 80$ plies
 - Exponential estimate: $b^d = 35^{80} \approx 10^{123}$
- Shannon’s Bound: In his seminal 1950 paper, Claude Shannon estimated the game-tree size of chess to be at least 10^{120}



Efficiency of Minimax Search with Alpha-Beta Pruning

Theorem (Donald Knuth, Ronald W. Moore (1975))

*On a uniform game tree with branching factor b and depth d , the **best-case** running time of minimax search with alpha-beta pruning is bounded by $O(b^{d/2}) = O(\sqrt{b^d})$, if every nodes' children are visited in the **perfect order***

- Perfect ordering (also known as the optimal move ordering):
 - Max nodes: you always try the move that leads to the highest minimax value first
 - Min nodes: you always try the move that leads to the lowest minimax value first
- Depth of search is effectively doubled in the ideal case
- For chess, we need to handle at least 10^{61} nodes of the game tree in the best-case and 10^{123} nodes when the nodes' children are visited in an arbitrary order



Evaluation Function

- Cut off search at a certain depth and compute the value of an **evaluation function** for a state instead of its minimax value
 - The evaluation function may be thought of as the probability of winning from a given state or the expected value of that state
 - Evaluation functions may be learned from game databases or by having the program play many games against itself



Evaluation Function

- Cut off search at a certain depth and compute the value of an **evaluation function** for a state instead of its minimax value
 - The evaluation function may be thought of as the probability of winning from a given state or the expected value of that state
 - Evaluation functions may be learned from game databases or by having the program play many games against itself
- The evaluation function employed by Deep Blue
 - Deep Blue evaluated non-terminal nodes at a fixed search depth using a *handcrafted* evaluation function
 - This function combined fast hardware-accelerated evaluation and a slower, more detailed software evaluation to score *material balance* (weight of each chess piece), pawn structure, king safety, piece mobility, and other positional features
 - It was initially written in a generalized form, with many to-be-determined parameters. Values for these parameters were determined by analyzing thousands of master games



Table of Contents

- ▶ Chess: Deep Blue
- ▶ Go: AlphaGo Zero
- ▶ Poker: Deep CFR



People's Daily's report on DeepBlue (May 13, 1997)

人机大战 了犹未了 ——记卡斯帕罗夫与“深蓝”的较量（上）

【作者：王烁】

【人民日报 1997年5月13日 第7版 国际】

【字号：加大 还原 减小】



本报记者 王 烁

5月11日，国际象棋世界冠军卡斯帕罗夫与IBM公司的国际象棋电脑“深蓝”的六局对抗赛降下帷幕。在前五局以2·5对2·5打平的情况下，卡斯帕罗夫在第六盘决胜局中仅走了19步就向“深蓝”拱手称臣。整场比赛进行了不到一个小时（见图 新华社记者 毛众役摄）。“深蓝”赢得了这场具有特殊意义的对抗。



People's Daily's report on DeepBlue (May 14, 1997)

记者采访了原中山大学化学系教授陈志行先生，他认为，这很难说，但是至少在很长一段时间内，我们不必担心会看到这种事情。一是因为目前电脑的运算速度还远远不够。以“深蓝”为例，它的研制者们曾经考虑过能不能让“深蓝”下围棋，很快他们就发现，对于目前的电脑速度来说，围棋是一个过于复杂的系统，于是放弃了这一计划。陈先生自1989年以来致力于电脑围棋程序设计，他所设计的软件“手谈”多次获得世界电脑围棋赛冠军，在电脑围棋领域里的地位与“深蓝”相仿，但是它至今还处于非常幼稚的水平。陈先生告诉记者，他希望在他的有生之年，将“手谈”的水平提高到他让九个子。可是，“我恐怕看不到这一天了。”

更重要的一点是现在人工智能的水平还很低。最主要的原因是，人类对自己的认识还不够深刻，要搞明白人类自己的智能是怎么回事，目前还说不清要多长时间，要在这基础上探索人工智能的前景，那就更是遥遥无期。陈志行先生说，他在互联网络上看到人们讨论要多少年以后电脑才可能与人脑在围棋领域相抗，有人说要1000年，陈先生作为专业人士表示了谨慎的乐观，他说：“100年吧。”



Prof. Zhixing Chen's HandTalk (GeoMate)

“The mid-1990s were dominated by the program HANDTALK, written by Zhixing Chen.”

— Jonathan Schaeffer, A Gamut of Games (cited by 130+ times)

“Handtalk was the best performing Go program from 1995 to 1997.”

— Aleksandr Shnayderman, State of the Art in Computer Games: Challenges and Solutions

- Handtalk implements its own Alpha-Beta search engine (written in IBM-PC assembly language)
 - Only a very small number (around 4) of candidate moves are evaluated using a static evaluation function
 - Searching depth: 11-ply (BASIC version)



Prof. Zhixing Chen's HandTalk (GeoMate)



- Why is Prof. Chen so negative in his predictions about the development of Go AI?



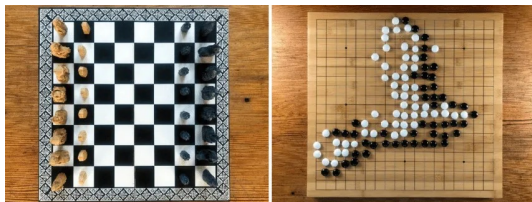
Complexity of Go

- **State-Space Complexity:** the number of legal positions
 - A 19×19 Go board has $19^2 = 361$ intersections, each of which can be empty, black, or white, giving 3^{361} total colorings, but most are illegal

Theorem (Tromp & Farnebäck, 2016)

The number of legal positions on a 19×19 Go board is about $2.08168199382 \times 10^{170}$

- By contrast, the state-space complexity of chess is in the order of 10^{46} to 10^{47}





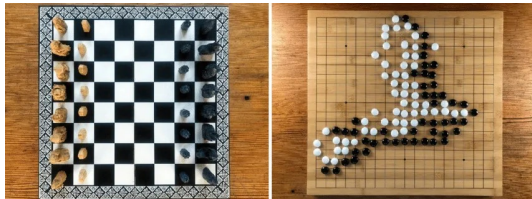
Complexity of Go

- **Game-Tree Complexity:** total nodes/leaves in the full game tree
 - Bounded by b^d
 - Expert-level Go games last on average ~ 150 moves, with ~ 250 legal choices per move
 - Giving the game-tree complexity of $250^{150} \approx 10^{360}$

Theorem (Tromp & Farbäck 2007)

Theoretically possible games: lower bound $10^{10^{48}}$, upper bound $10^{10^{171}}$

- Recall the game-tree complexity of chess is bounded by 10^{123}





Complexity of Go

- The state space of Go is much, much larger than chess
- Calls for more efficient searching algorithms





Monte Carlo Tree Search: MDP Model

- **Monte Carlo Tree Search** (*MCTS*) is a policy-optimization algorithm for finite **Markov Decision Processes** (MDPs) based on random episode sampling



Monte Carlo Tree Search: MDP Model

- **Monte Carlo Tree Search** (*MCTS*) is a policy-optimization algorithm for finite **Markov Decision Processes** (MDPs) based on random episode sampling
- An MDP consists of the following components
 - State space X
 - Action space A
 - Transition probability function $P : X \times A \times X \mapsto [0, 1]$
 - Reward function $R : X \times A \mapsto [0, 1]$
 - Decision policy $\pi : X \mapsto \Delta(A)$
- Optimal state value function: $V^*(x) = \max_{a \in A} Q^*(x, a)$
- Optimal state-action value function: $Q^*(x, a) = R(x, a) + \sum_{x' \in X} P(x, a, x') V^*(x')$



Monte Carlo Tree Search: MDP Model

- **Monte Carlo Tree Search** (*MCTS*) is a policy-optimization algorithm for finite **Markov Decision Processes** (MDPs) based on random episode sampling
- An MDP consists of the following components
 - State space X
 - Action space A
 - Transition probability function $P : X \times A \times X \mapsto [0, 1]$
 - Reward function $R : X \times A \mapsto [0, 1]$
 - Decision policy $\pi : X \mapsto \Delta(A)$
- Converted to a minimax game tree
 - Game states S , our actions A , opponent's actions O
 - Transition probability $P : S \times A \times O \times S \mapsto [0, 1]$
 - Reward $R : S \times A \times O \mapsto [0, 1]$



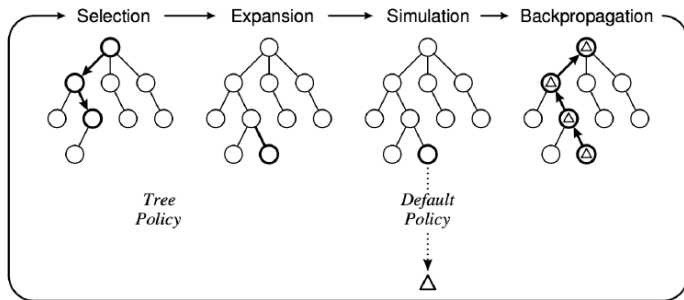
Monte Carlo Tree Search: MDP Model

- **Monte Carlo Tree Search (MCTS)** is a policy-optimization algorithm for finite **Markov Decision Processes (MDPs)** based on random episode sampling
- An MDP consists of the following components
 - State space X
 - Action space A
 - Transition probability function $P : X \times A \times X \mapsto [0, 1]$
 - Reward function $R : X \times A \mapsto [0, 1]$
 - Decision policy $\pi : X \mapsto \Delta(A)$
- Converted to a minimax game tree
 - Optimal state value function: $V^*(s) = \max_{\pi} \min_{o \in O} \mathbb{E}_{a \sim \pi(s)} [Q^*(s, a, o)]$
 - Optimal state-action value function:
$$Q^*(s, a, o) = R(s, a, o) + \sum_{s' \in S} P(s, a, o, s') \cdot V^*(s')$$



Monte Carlo Tree Search: Algorithm

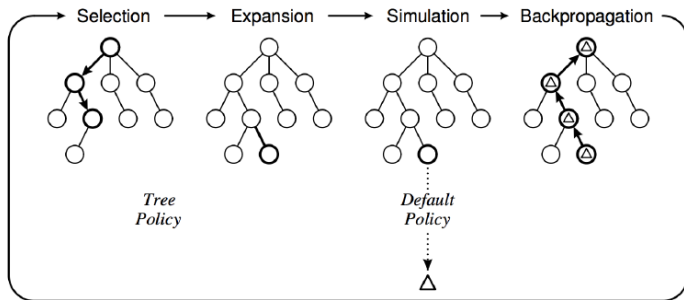
- Overview: MCTS is a **heuristic** search algorithm using Monte Carlo simulations
- Until exhaustion of a user-specified time budget, MCTS iteratively samples full episodes, expanding the decision tree incrementally
- Four phases:
 1. **Selection:** Traverse the tree from the root to a leaf using a *tree policy*





Monte Carlo Tree Search: Algorithm

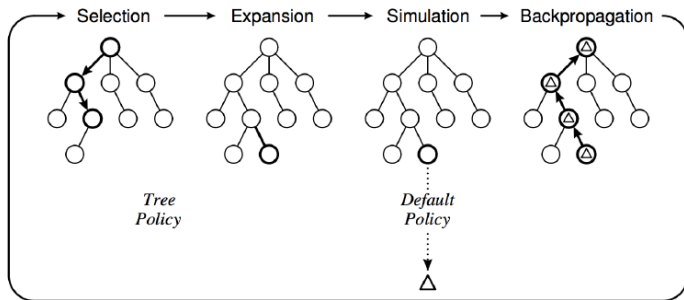
- Overview: MCTS is a **heuristic** search algorithm using Monte Carlo simulations
- Until exhaustion of a user-specified time budget, MCTS iteratively samples full episodes, expanding the decision tree incrementally
- Four phases:
 - 1 **Selection**: Traverse the tree from the root to a leaf node, selecting nodes based on a policy.
 - 2 **Expansion**: Add one or more child nodes to the selected leaf node if it is not fully expanded.
 - 3 **Simulation**: Simulate the game from the newly added node to a terminal state using a default policy.
 - 4 **Backpropagation**: Propagate the result from the terminal state back up the tree, updating the statistics of the nodes along the path.





Monte Carlo Tree Search: Algorithm

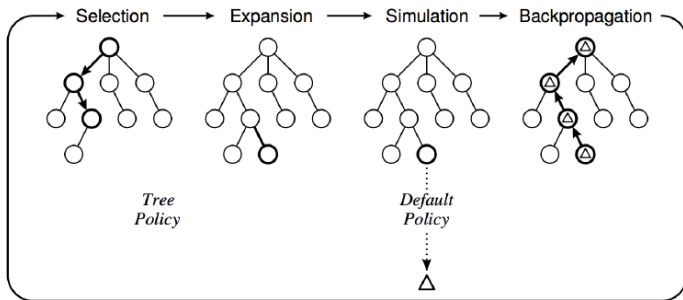
- Overview: MCTS is a **heuristic** search algorithm using Monte Carlo simulations
- Until exhaustion of a user-specified time budget, MCTS iteratively samples full episodes, expanding the decision tree incrementally
- Four phases:
 - 3 **Simulation:** Run a random playout (rollout) from the new node to estimate its value





Monte Carlo Tree Search: Algorithm

- Overview: MCTS is a **heuristic** search algorithm using Monte Carlo simulations
- Until exhaustion of a user-specified time budget, MCTS iteratively samples full episodes, expanding the decision tree incrementally
- Four phases:
 - 4 **Backpropagation**: Update node statistics along the path back to the root





Monte Carlo Tree Search: Algorithm

- Overview: MCTS is a **heuristic** search algorithm using Monte Carlo simulations
- Until exhaustion of a user-specified time budget, MCTS iteratively samples full episodes, expanding the decision tree incrementally
- Four phases:
 - 4 **Backpropagation**: Update node statistics along the path back to the root
 - $R_{x,a}$: Total reward observed for all children of x connected by a branch for a
 - $N_{x,a}$: Total number observed of transition to children of x by selecting a
 - $R_{x,a}/N_{x,a}$: Approximate $Q_{x,a}$



Monte Carlo Tree Search: Pseudocode

Algorithm 1 Monte Carlo Tree Search

```
1: procedure MONTECARLOTREESearch( $x_0$ )  
2:    $\tau_0 \leftarrow \text{MakeTree}(x_0)$  ▷ Initialize decision tree with root state  $x_0$   
3:   repeat  
4:      $(\tau_i, r) \leftarrow \text{TreePolicy}(\tau_0)$  ▷ Exploration, returning new leaf and current reward  
5:      $x_i \leftarrow \tau_i.\text{state}$   
6:      $r \leftarrow \text{DefaultPolicy}(x_i)$  ▷ Simulation, completing episode and returning total  
reward  
7:      $\text{Backpropagate}(\tau_i, r)$  ▷ Update node statistics along exploration path  
8:   until Timeout()  
9:   return BestAction( $\tau_0$ ) ▷ Pick the approximately optimal root-level action  
10: end procedure
```



MCTS: Reduction to Multi-Armed Bandits

- MCTS uses a simple timeout as its stopping condition
- Desirable properties of the **selection** phase:
 - The probability of selecting a sub-optimal decision quickly approaches zero
 - The probability of a sub-optimal decision converges to zero given unbounded time
- Keypoint: Balancing the **Exploration-Exploitation** trade-off



MCTS: Reduction to Multi-Armed Bandits

- MCTS uses a simple timeout as its stopping condition
- Desirable properties of the **selection** phase:
 - The probability of selecting a sub-optimal decision quickly approaches zero
 - The probability of a sub-optimal decision converges to zero given unbounded time
- Keypoint: Balancing the **Exploration-Exploitation** trade-off

Multi-Arm Bandits (MAB)

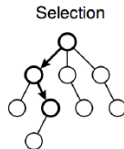
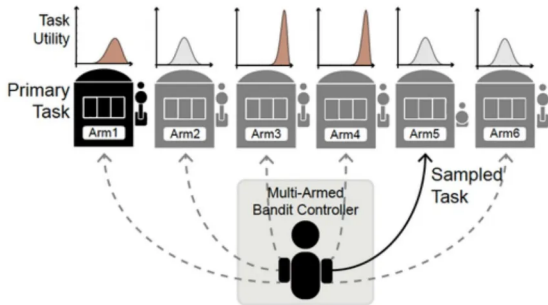
- Given K arms; each arm a is associated with an unknown distribution $\mathcal{D}(a)$ with mean $Q(a)$
- In each step $t \geq 1$, the player chooses an arm a_t and receives the reward r_t drawn from $\mathcal{D}(a_t)$ randomly and independently
- The player aims to maximize her accumulated reward $\sum_{t \in [T]} r_t$
- **Regret:**

$$\max_{a^* \in [K]} \mathbb{E}[Q(a^*)] \cdot T - \sum_{t \in [T]} r_t = \sum_{t \in [T]} \max_{a^* \in [K]} \mathbb{E}[Q(a^*)] - \sum_{t \in [T]} Q(a_t)$$



MCTS: Reduction to Multi-Armed Bandits

- MCTS uses a simple timeout as its stopping condition
- Desirable properties of the **selection** phase:
 - The probability of selecting a sub-optimal decision quickly approaches zero
 - The probability of a sub-optimal decision converges to zero given unbounded time
- Keypoint: Balancing the **Exploration-Exploitation** trade-off





MCTS: Reduction to Multi-Armed Bandits

- MCTS uses a simple timeout as its stopping condition
- Desirable properties of the **selection** phase:
 - The probability of selecting a sub-optimal decision quickly approaches zero
 - The probability of a sub-optimal decision converges to zero given unbounded time
- Keypoint: Balancing the **Exploration-Exploitation** trade-off

Multi-Arm Bandits (MAB)

— **Regret:**

$$\sum_{t \in [T]} \max_{a^* \in [K]} \mathbb{E}[Q(a^*)] - \sum_{t \in [T]} Q(a_t)$$

- Best possible regret: $\Omega(\ln T)$
- Any MAB algorithm with a **sublinear** regret possesses the two desirable properties mentioned above (**why?**)



UCB (UCB1) for MAB

- UCB: a general family of MAB algorithm frameworks
 - Estimates the **Upper Confidence Bound** $U_t(a)$ for each arm a such that $U_t(a) > Q(a)$ with *high probability*
 - With high confidence, our estimate $U_t(a)$ is an upper bound on the true arm value $Q(a)$
 - Selects the arm a_t that maximizes the estimate $U_t(a_t)$



UCB (UCB1) for MAB

- UCB: a general family of MAB algorithm frameworks
 - Estimates the **Upper Confidence Bound** $U_t(a)$ for each arm a such that $U_t(a) > Q(a)$ with *high probability*
 - With high confidence, our estimate $U_t(a)$ is an upper bound on the true arm value $Q(a)$
 - Selects the arm a_t that maximizes the estimate $U_t(a_t)$
- UCB1: a specific instance of the more general UCB family
 - Tracks the *empirical average* of each arm's rewards $\hat{Q}_t(a) = \sum_{t:a_t=a} r_t / |t : a_t = a|$
 - Set $U_t(a) = \hat{Q}_t(a) + \sqrt{\frac{2 \ln t}{n_t(a)}}$, where $n_t(a) = |t : a_t = a|$
 - The term $\sqrt{\frac{2 \ln t}{n_t(a)}}$ is an “**optimism bonus**”
 - By adding an optimism bonus to under-explored actions, UCB encourages *exploring* options that might yield higher rewards while still *exploiting* known high-reward actions



Basic Ideas of UCB

- If $U_t(a) > Q(a)$ holds for all t and a , we have $U_t(a_t) \geq U_t(a^*) > Q(a^*)$. This implies that

$$\begin{aligned}\sum_{t \in [T]} Q(a^*) - Q(a_t) &= \sum_{t \in [T]} U_t(a_t) - Q(a_t) + Q(a^*) - U_t(a_t) \\ &\leq \sum_{t \in [T]} U_t(a_t) - Q(a_t) \\ &= \sum_{t \in [T]} \hat{Q}_t(a_t) + b_t(a_t) - Q(a_t)\end{aligned}$$

- We have to choose proper bonus b_t so that
 - $\sum_{t \in [T]} b_t$ is sublinear in T (requiring small bonus)
 - $U_t(a) > Q(a)$ holds for all t and a with a high probability (requiring large bonus)



Basic Ideas of UCB1

Theorem (Chernoff-Hoeffding Bound)

Let $X_1, X_2, \dots, X_n$ be i.i.d. random variables such that $X_i \in [0, 1]$ and let $\hat{X}_n = \frac{1}{n} \sum_{i=1}^n X_i$ be the sample mean. Then

$$\Pr \left[E[X] > \hat{X}_n + u \right] < \exp(-2nu^2)$$

- Let $b = \sqrt{\frac{2 \ln t}{n_t(a)}}$, the inequality $U_t(a) \leq Q(a)$ holds with probability

$$\Pr \left[Q(a) > \hat{Q}_t(a) + b \right] < \exp(-2 \cdot n_t(a) \cdot \frac{2 \ln t}{n_t(a)}) = \frac{1}{t^4}$$

- Taking the union bound, we have $U_t(a) > Q(a)$ for all $t \in [T]$ and $a \in [K]$ with probability $1 - O(K/T^3)$



Regret Analysis of UCB1

- For each arm a , let $\Delta_a = Q(a^*) - Q(a)$, then the regret of UCB1 is bounded by

$$\sum_{a:\Delta_a>0} \frac{8 \ln T}{\Delta_a} + \left(1 + \frac{\pi^2}{3}\right) \Delta_a$$

- How about the difficult cases where Δ_a is not well-bounded?



Regret Analysis of UCB1

- For each arm a , let $\Delta_a = Q(a^*) - Q(a)$, then the regret of UCB1 is bounded by

$$\sum_{a:\Delta_a>0} \frac{8 \ln T}{\Delta_a} + \left(1 + \frac{\pi^2}{3}\right) \Delta_a$$

- How about the difficult cases where Δ_a is not well-bounded?

Theorem (Auer, Cesa-Bianchi, and Paul Fischer, 2002)

The worst-case upper bound on the regret of UCB1 is $O(KT \ln T)$



UCT Selection Phase with UCB1

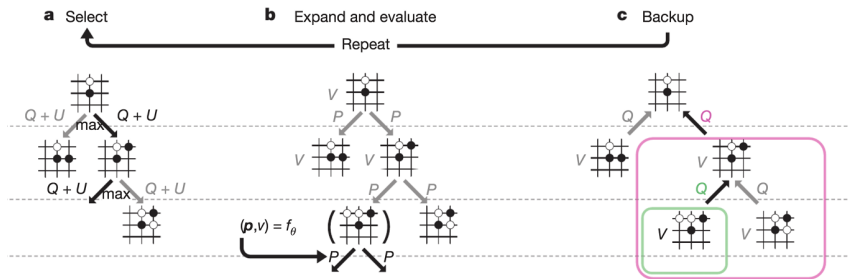
Algorithm 2 UCT Action Selection

```
1: procedure SELECTACTION(position, depth)
2:    $maxv \leftarrow -\infty$ 
3:   for  $i$  in 1 to  $nActions$  do
4:     if position.actions[ $i$ ]. $n = 0$  then
5:        $v \leftarrow +\infty$ 
6:     else
7:        $v \leftarrow$  position.actions[ $i$ ].value +  $\sqrt{\frac{2 \cdot \ln(nsum)}{\text{position.actions}[i].n}}$ 
8:     end if
9:     if  $v > maxv$  then
10:       $maxv \leftarrow v; j \leftarrow i$ 
11:    end if
12:  end for
13:  return position.actions[ $j$ ]
14: end procedure
```



AlphaGo Zero: MCTS

- **AlphaGo Zero** is a specific instantiation of the general MCTS algorithm
- Iteratively builds up a game tree
 1. Using a Tree Policy to decide a path down the known tree
 2. A leaf node is expanded and its value estimated using a trained NN
 3. Information is percolated back





AlphaGo Zero's Tree Policy: PUCT

- In episode i , Tree Policy chooses the action $a_t = \operatorname{argmax}_a \{\hat{Q}(s_t, a) + b(s_t, a)\}$ where

$$b(s, a) = cP(s, a) \cdot \frac{\sqrt{\sum_{a'} N(s, a')}}{1 + N(s, a)} \quad (1)$$

- The bonus Eq. (1) is *based* on PUCT, a variance of UCT



AlphaGo Zero's Tree Policy: PUCT

- In episode i , Tree Policy chooses the action $a_t = \operatorname{argmax}_a \{\hat{Q}(s_t, a) + b(s_t, a)\}$ where

$$b(s, a) = cP(s, a) \cdot \frac{\sqrt{\sum_{a'} N(s, a')}}{1 + N(s, a)} \quad (1)$$

- The bonus Eq. (1) is *based* on PUCT, a variance of UCT

PUCB (Rosin, 2009)

- Predictor + UCB
 - The predictor gives a weight vector M over the arms
 - PUCB uses additive penalties proportional to $1/M(a)$
 - Chooses the arm $a_t = \operatorname{argmax}_a \hat{Q}_t(a) + \sqrt{\frac{3 \log t}{2n_a(t)}} - \frac{2}{M(a)} \sqrt{\frac{\log t}{t}}$
 - Theorem: The regret of PUCB is bounded by $\frac{1}{M(a^*)} \sqrt{T \log T}$



AlphaGo Zero's Expansion: Neural Network

- How does AGZ determines $\hat{Q}(s, a)$ and $P(s, a)$?
 - The vanilla MCTS uses the Monte-Carlo sampling (random rollouts)



AlphaGo Zero's Expansion: Neural Network

- How does AGZ determines $\hat{Q}(s, a)$ and $P(s, a)$?
 - The vanilla MCTS uses the Monte-Carlo sampling (random rollouts)
- AGZ uses an NN f_θ when expanding a leaf node s
 - Outputs $f_\theta(s) = (\mathbf{p}, v)$
 - The distribution $\mathbf{p}$ is used as $P(s, a)$
 - The value v is propagated backward to compute the empirical average payoff $\hat{Q}(s, a)$



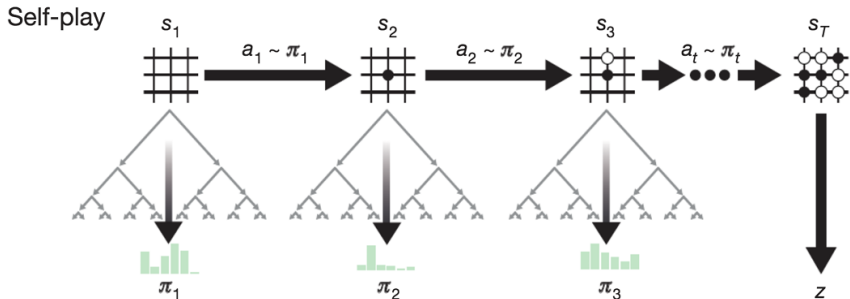
AlphaGo Zero's Expansion: Neural Network

- How does AGZ determines $\hat{Q}(s, a)$ and $P(s, a)$?
 - The vanilla MCTS uses the Monte-Carlo sampling (random rollouts)
- AGZ uses an NN f_θ when expanding a leaf node s
 - Outputs $f_\theta(s) = (\mathbf{p}, v)$
 - The distribution $\mathbf{p}$ is used as $P(s, a)$
 - The value v is propagated backward to compute the empirical average payoff $\hat{Q}(s, a)$
- The **hyperparameters** θ is optimized using *Stochastic Gradient Decent* (SGD) using the he set of moves from the last 500000 games of **self-play**
 - Previous versions of AlphaGo required a database of games from expert human players
 - AlphaGo Zero plays games against itself to learn Go strategies



AlphaGo Zero's NN: Training Set

- AGZ builds the training set by recording the game sequence $\{(s_t, \pi_t, z_t)\}_{t \in [T]}$, where $z_t \in 1, -1$ indicates if the current player wins/loses





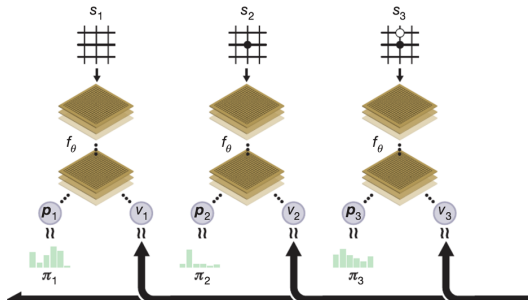
AlphaGo Zero's NN: SGD

- The loss function on a training example (s, π, z) is

$$\mathcal{L}(\theta; s, \pi, z) = (z - v)^2 - \pi \log \mathbf{p} + c \|\theta\|^2 \quad (2)$$

- SGD minimizes the loss by updating the hyperparameters as follows

$$\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} \mathcal{L}(\theta; s, \pi, z)$$





Results

- After 36 hours of training, AlphaGo Zero was able to beat AlphaGo Lee
- After 3 days of training, AlphaGo Zero was able to beat AlphaGo Lee 100 games to 0
 - AlphaGo Zero was run on one machine with 4 Tensor Processing Units (TPUs)
 - AlphaGo Lee was run in a distributed manner using 48 TPUs in total

Lee



AlphaGo Lee



ALPHAGO ZERO





Table of Contents

- ▶ Chess: Deep Blue
- ▶ Go: AlphaGo Zero
- ▶ Poker: Deep CFR



Poker & Games with Imperfect Information





Extensive-form Game with Imperfect Information

- **Extensive-form Games:** A specification of a game allowing for the explicit representation of key aspects:
 - Sequencing of players' possible moves
 - Players' choices at every decision point
 - The (imperfect) information each player has about the other player's moves
 - Their payoffs for all possible game outcome
 - Different from *Normal-form Games*: Represented by a payoff-matrix

		Player 2			
		Oo, Fo	Oo, Ff	Of, Fo	Of, Ff
Player 1	O	3, 2	3, 2	0, 0	0, 0
	F	0, 2	2, 3	0, 0	2, 3

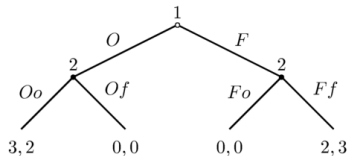


Figure: Battle of Sex: normal-form with perfect information

Figure: Battle of Sex: extensive-form with perfect information



Extensive-form Game with Imperfect Information

- **Extensive-form Games:** A specification of a game allowing for the explicit representation of key aspects:
 - Sequencing of players' possible moves
 - Players' choices at every decision point
 - The (imperfect) information each player has about the other player's moves
 - Their payoffs for all possible game outcome
 - Different from *Normal-form Games*: Represented by a payoff-matrix

		Player 2	
		o	f
Player 1	o	3, 2	0, 0
	F	0, 0	2, 3

Figure: Battle of Sex: normal-form with imperfect information

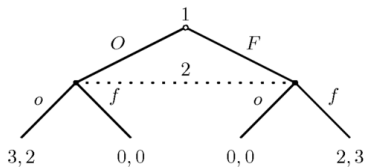


Figure: Battle of Sex: extensive-form with imperfect information



Extensive-form Game with Imperfect Information

- Components in an Extensive-form Game with Imperfect Information (IEIG)
 - A finite set of n (rational) players
 - A rooted tree, called the game tree
 - Each terminal (leaf) node of the game tree has an n -tuple of payoffs
 - A partition of the non-terminal nodes of the game tree in $n + 1$ subsets, one for each (rational) player, and with a special subset for a fictitious player called Chance
 - Each node of the Chance player has a probability distribution over its outgoing edges
 - **Imperfect Information:** Each set of nodes of a rational player is further partitioned into **information sets**
 - An information set I is a subset of game-tree nodes that are indistinguishable to a player
 - There is a one-to-one correspondence between outgoing edges of any two nodes of the same information set



Game Theoretic Definitions

- Strategy: a strategy σ_i of player i assigns a distribution over $A(I)$ for each $I \in \mathcal{I}_i$
- Strategy profile: $\sigma = \{\sigma_i\}_{i \in [n]}$ consists of strategies of all the players
 - We use σ_{-i} to represent the profile of strategies of players other than i



Game Theoretic Definitions

- Strategy: a strategy σ_i of player i assigns a distribution over $A(I)$ for each $I \in \mathcal{I}_i$
- Strategy profile: $\sigma = \{\sigma_i\}_{i \in [n]}$ consists of strategies of all the players
 - We use σ_{-i} to represent the profile of strategies of players other than i
- Reach Probability: $\pi^\sigma(h)$ refers to the probability of reaching state h with all players following σ
 - $\pi_i^\sigma(h)$ and $\pi_{-i}^\sigma(h)$: the contribution of player i to this probability and the contribution of players other than i , respectively



Game Theoretic Definitions

- Strategy: a strategy σ_i of player i assigns a distribution over $A(I)$ for each $I \in \mathcal{I}_i$
- Strategy profile: $\sigma = \{\sigma_i\}_{i \in [n]}$ consists of strategies of all the players
 - We use σ_{-i} to represent the profile of strategies of players other than i
- Reach Probability: $\pi^\sigma(h)$ refers to the probability of reaching state h with all players following σ
 - $\pi_i^\sigma(h)$ and $\pi_{-i}^\sigma(h)$: the contribution of player i to this probability and the contribution of players other than i , respectively
- Utility $u_i : Z \mapsto \mathbb{R}$: the function $u_i(z)$ returns how much player i wins at leaf $z \in Z$
- Expected payoff: $u_i(\sigma) = \sum_{h \in Z} u_i(h) \cdot \pi^\sigma(h)$



Game Theoretic Definitions

- Strategy: a strategy σ_i of player i assigns a distribution over $A(I)$ for each $I \in \mathcal{I}_i$
- Strategy profile: $\sigma = \{\sigma_i\}_{i \in [n]}$ consists of strategies of all the players
 - We use σ_{-i} to represent the profile of strategies of players other than i
- Reach Probability: $\pi^\sigma(h)$ refers to the probability of reaching state h with all players following σ
 - $\pi_i^\sigma(h)$ and $\pi_{-i}^\sigma(h)$: the contribution of player i to this probability and the contribution of players other than i , respectively
- Utility $u_i : Z \mapsto \mathbb{R}$: the function $u_i(z)$ returns how much player i wins at leaf $z \in Z$
- Expected payoff: $u_i(\sigma) = \sum_{h \in Z} u_i(h) \cdot \pi^\sigma(h)$
- **Nash Equilibrium** (NE): a strategy profile σ is a NE iff. no player can improve her payoff by deviation

$$u_i(\sigma_i, \sigma_{-i}) \geq u_i(\sigma'_i, \sigma_{-i}) \quad \forall i \in [n], \sigma'_i \in S_i$$



Approximate Nash Equilibrium and Regret

Approximate Nash Equilibrium

A strategy profile σ is an approximate Nash equilibrium iff. no player can increase her payoff by a constant $\epsilon > 0$ through deviation

$$u_i(\sigma_i, \sigma_{-i}) + \epsilon \geq u_i(\sigma'_i, \sigma_{-i}) \quad \forall i \in [n], \sigma'_i \in S_i$$



Approximate Nash Equilibrium and Regret

Approximate Nash Equilibrium

A strategy profile σ is an approximate Nash equilibrium iff. no player can increase her payoff by a constant $\epsilon > 0$ through deviation

$$u_i(\sigma_i, \sigma_{-i}) + \epsilon \geq u_i(\sigma'_i, \sigma_{-i}) \quad \forall i \in [n], \sigma'_i \in S_i$$

Regret in IIEG

In an extensive-form game with imperfect information, the *average* regret of player i is defined to be

$$R_i^T = \frac{1}{T} \max_{\sigma_i^* \in \Sigma_i} \sum_{t=1}^T (u_i(\sigma_i^*, \sigma_{-i}^t) - u_i(\sigma^t))$$

- Question: How to compute the strategy of each player with a vanishing regret?



Approximate Nash Equilibrium and Regret

Theorem (Waugh, 2009)

In a two-player zero-sum game, if both players' average regret is bounded by ϵ , then their average strategies $\langle \bar{\sigma}_1, \bar{\sigma}_2 \rangle$ form a 2ϵ -approximate Nash equilibrium, where

$$\bar{\sigma}_i(I) = \frac{\sum_{t \in [T]} \pi_i^{\sigma^t}(I) \cdot \sigma_i^t(I)}{\sum_{t \in [T]} \pi_i^{\sigma^t}(I)}$$

- Question: How to compute the strategy of each player with a vanishing regret?



Regret and Counterfactual Regret

Counterfactual Regret

For player i , action a at information set I , the **counterfactual** regret at time T is:

$$\tilde{R}_i^T(I, a) = \sum_{h \in I} \pi_{-i}^\sigma(h) [v_i(a, h) - v_i(\sigma, h)]$$

where the *counterfactual value* v_i is defined to be

$$v_i(\sigma, h) = \sum_{z \in Z} \pi_{-i}^\sigma(h) \pi_i^\sigma(h, z) \cdot u_i(z) .$$

It refers to the expected payoff to i when reaching h , weighted by the probability that i would reach i if she tried to do so



Regret and Counterfactual Regret

Theorem (Zinkevich et al. 2007)

The overall regret R_i^T is bounded by the sum of counterfactual regrets:

$$R_i^T \leq \sum_{I \in \mathcal{I}_i} \max_{a \in A(I)} \tilde{R}_i^T(I, a)$$

- Implication: Minimizing cumulative counterfactual regret minimizes overall regret
- Leads to convergence to an ϵ -approximate Nash equilibrium



Counterfactual Regret Minimization: Regret Matching

- Strategy update uses Blackwell's Approachability Theorem via regret matching:

$$\sigma_i^{T+1}(I, a) = \begin{cases} \frac{\tilde{R}_i^{T,+}(I, a)}{\sum_{a' \in A(I)} \tilde{R}_i^{T,+}(I, a')} & \text{if } \sum_{a' \in A(I)} \tilde{R}_i^{T,+}(I, a') > 0 \\ \frac{1}{|A(I)|} & \text{otherwise} \end{cases}$$

- $\tilde{R}_i^{T,+}(I, a) = \max\{\tilde{R}_i^T(I, a), 0\}$: Positive counterfactual regret
- Actions are selected proportional to positive CFR; otherwise, choose randomly

Theorem (Cesa-Bianchi & Lugosi, 2006; Zinkevich et al. (2007))

If a player plays according to regret matching in infoset I on every iteration, then on iteration T ,

$$R_i^T \leq \Delta_{u,i} \cdot \sqrt{\frac{|A_i|}{T}}$$



Vanilla CFR Algorithm

- Initialize strategy profile σ with uniform random probabilities
- Iterate:
 1. Update counterfactual values via self-play
 2. Propagate values through terminal histories, weighted by reach probability
 3. Update strategy profile using regret matching
- Requires traversing the **entire game tree** each iteration



Vanilla CFR Algorithm

Algorithm 3 Vanilla Counterfactual Regret Minimization

```
function CFR( $h, \pi_1, \pi_2, t$ )  
   $I \leftarrow$  information set of  $h$ ;  $v \leftarrow 0$ ;  $v_a \leftarrow$  array of size  $|A(I)|$   
  for each  $a \in A(I)$  do  
     $h' \leftarrow$  next state after action  $a$  at  $h$   
     $v_a[a] \leftarrow$  CFR( $h', \pi_1 \cdot \sigma_1(I, a), \pi_2, t$ ) if player 1  
     $v_a[a] \leftarrow$  CFR( $h', \pi_1, \pi_2 \cdot \sigma_2(I, a), t$ ) if player 2  
     $v \leftarrow v + \sigma(I, a) \cdot v_a[a]$   
  end for  
  for each  $a \in A(I)$  do  
     $R_i^T(I, a) \leftarrow R_i^T(I, a) + \pi_{-i} \cdot (v_a[a] - v)$   
    Update  $\sigma_i^{T+1}(I, a)$  using regret matching  
  end for  
  return  $v$   
end function
```



Monte Carlo Counterfactual Regret Minimization

- MCCFR: Avoids traversing the entire game tree while still having the immediate counterfactual regrets be unchanged in expectation
 - Restricting the terminal histories that we consider on each iteration



Monte Carlo Counterfactual Regret Minimization

- MCCFR: Avoids traversing the entire game tree while still having the immediate counterfactual regrets be unchanged in expectation
 - Restricting the terminal histories that we consider on each iteration
- Key ideas of MCCFR:
 - **Produce** a set of *blocks* $Q_1, Q_2, \dots, Q_r$ for the leaf set Z
 - In each iteration, sample one of the blocks Q_j with probability q_j , and only consider Q_j in the current iteration
 - Replace the counterfactual value with **sampled counterfactual value**:

$$v_i(\sigma, h|j) = \sum_{z \in Q_j} \frac{1}{q(z)} \pi_{-i}^{\sigma}(h) \pi_i^{\sigma}(h, z) \cdot u_i(z)$$



Monte Carlo Counterfactual Regret Minimization

- MCCFR: Avoids traversing the entire game tree while still having the immediate counterfactual regrets be unchanged in expectation
 - Restricting the terminal histories that we consider on each iteration
- Key ideas of MCCFR:
 - **Produce** a set of *blocks* $Q_1, Q_2, \dots, Q_r$ for the leaf set Z
 - In each iteration, sample one of the blocks Q_j with probability q_j , and only consider Q_j in the current iteration
 - Replace the counterfactual value with **sampled counterfactual value**:

$$v_i(\sigma, h|j) = \sum_{z \in Q_j} \frac{1}{q(z)} \pi_{-i}^{\sigma}(h) \pi_i^{\sigma}(h, z) \cdot u_i(z)$$

- MCCFR with external sampling (Lanctot et al. 2009): $q(z) = \pi_{-i}^{\sigma}(z)$
 - Average regret $R_i^T \leq \left(1 + \sqrt{\frac{2}{p}}\right) |\mathcal{I}_i| \Delta_{u,i} \sqrt{\frac{|A_i|}{T}}$



Deep CFR: Motivation

- Counterfactual Regret Minimization (CFR) is the leading framework for solving large imperfect-information games
- Large games require abstraction to simplify states
 - Abstraction is often manual, domain-specific, and may miss strategic nuances
- Abstraction requires equilibrium knowledge, creating a chicken-and-egg problem



Deep CFR: Motivation

- Counterfactual Regret Minimization (CFR) is the leading framework for solving large imperfect-information games
- Large games require abstraction to simplify states
 - Abstraction is often manual, domain-specific, and may miss strategic nuances
- Abstraction requires equilibrium knowledge, creating a chicken-and-egg problem
- Deep CFR eliminates abstraction by using deep neural networks to approximate CFR behavior in the full game
 - Requiring minimal domain knowledge
- Goal: Achieve strong performance in large games like poker without **tabular CFR** or abstraction
 - Inspired by deep reinforcement learning successes in large state spaces



Deep CFR Technical Framework

- **Deep CFR Framework:**

- Uses deep neural networks for function approximation to mimic tabular CFR on unabstracted games
- Conducts K partial game tree traversals per iteration using external sampling MCCFR
- Value network $V(I, a|\theta_p^{t-1})$ predicts regrets; policy network Π approximates average strategy
- Regrets stored in reservoir-sampled memory $\mathcal{M}_{V,p}$; strategies in $\mathcal{M}_{\Pi}$
- Trains networks from scratch each iteration to minimize MSE loss



Deep CFR Technical Framework

- **Deep CFR Framework:**

- Uses deep neural networks for function approximation to mimic tabular CFR on unabstracted games
- Conducts K partial game tree traversals per iteration using external sampling MCCFR
- Value network $V(I, a|\theta_p^{t-1})$ predicts regrets; policy network Π approximates average strategy
- Regrets stored in reservoir-sampled memory $\mathcal{M}_{V,p}$; strategies in $\mathcal{M}_{\Pi}$
- Trains networks from scratch each iteration to minimize MSE loss

- **Differences from MCCFR:**

- MCCFR samples a subset of nodes, tracking sampled regrets, requiring full regret computation in tabular form
- Deep CFR approximates regrets across infosets using NN, avoiding tabular storage



Theoretical Guarantees of Deep CFR

- **Theorem 1:** For T iterations, K traversals per iteration, and max actions $|A|$, if value memory is large and MSE loss $\mathcal{L}_V^t - \mathcal{L}_{V^*}^t \leq \epsilon_{\mathcal{L}}$, total regret is bounded with probability $1 - \rho$:

$$R_p^T \leq \left(1 + \frac{\sqrt{2}}{\sqrt{\rho K}}\right) \Delta |\mathcal{I}_p| \sqrt{|A|/T} + 4 |\mathcal{I}_p| \sqrt{|A| \Delta \epsilon_{\mathcal{L}}}$$

where Δ is payoff range, $|\mathcal{I}_p|$ is number of infosets for player p .

- **Corollary 1:** As $T \rightarrow \infty$, average regret is bounded by:

$$4 |\mathcal{I}_p| \sqrt{|A| \Delta \epsilon_{\mathcal{L}}}$$

with high probability.

- **Implication:** Deep CFR converges to an ϵ -Nash equilibrium, with regret dependent on function approximation error.



Deep CFR Algorithm

Algorithm 4 Deep Counterfactual Regret Minimization

function DEEPCFR

Initialize advantage network $V(I, a|\theta_p)$ with θ_p to return 0 for all inputs.

Initialize reservoir-sampled memories $\mathcal{M}_{V,1}, \mathcal{M}_{V,2}, \mathcal{M}_{\Pi}$.

for CFR iteration $t = 1$ to T **do**

for each player p **do**

for traversal $k = 1$ to K **do**

 TRAVERSE($\emptyset, p, \theta_1, \theta_2, \mathcal{M}_{V,p}, \mathcal{M}_{\Pi}$)

 ▷ Collect data via external sampling

end for

 Train θ_p from scratch on loss $\mathcal{L}(\theta_p) = \mathbb{E}_{(I, t', \tilde{r}^{t'}) \sim \mathcal{M}_{V,p}} \left[t' \sum_a (\tilde{r}^{t'}(a) - V(I, a|\theta_p))^2 \right]$

end for

end for

 Train θ_{Π} on loss $\mathcal{L}(\theta_{\Pi}) = \mathbb{E}_{(I, t', \sigma^{t'}) \sim \mathcal{M}_{\Pi}} \left[t' \sum_a (\sigma^{t'}(a) - \Pi(I, a|\theta_{\Pi}))^2 \right]$

return θ_{Π}

end function



Experimental Results and Highlights

- **Heads-Up Limit Texas Hold'em (HULH):**
 - Deep CFR beats NFSP by $+43 \pm 2$ mbb/g and loses to a $3.3 \cdot 10^8$ bucket abstraction by only -11 ± 2 mbb/g
 - Competitive with 2007 Polaris AI, which used $3 \cdot 10^8$ buckets and beat humans by 52 mbb/g
- **Ablations:** Training from scratch and reservoir sampling reduce exploitability by 50% compared to fine-tuning or sliding-window memory
- **Conclusion:** Deep CFR is the first non-tabular CFR variant successful in large games, requiring minimal domain knowledge

Intelligent Heuristics in Games

Thank you for listening!

Any questions?